



FEBAUTO Racing

Competitor setup: Linux

- Part I** Install, clone, first drive
- Part II** The starter driver, scoring, looking inside
- Appendix** The starter package, file by file, and how the driver works

Formula Electric at Berkeley
September 2026, platform release v0.1.0

<https://github.com/Pranman1/feb-racing>
<https://pranman1.github.io/feb-racing/>

Contents

1	Install, once	2
2	Clone and set up	2
3	Drive manually	2
4	Drive autonomously with the starter kit	3
5	Score and post	3
6	Looking inside: extra terminals and Foxglove	3
7	When something goes wrong	4
8	Keeping up to date	4
9	What the car gives and takes	5
A	The starter package, file by file	5
B	How the driver drives, one scan at a time	6

What you are setting up

A simulator of the RoboRacer 1:10 car runs natively on your machine. Your code runs in a Docker container that carries ROS 2 and the bridge to the simulator. You never install ROS on the machine itself. Foxglove shows you the car's sensors live.

feb-racing/	the repo, your clone
feb-sim	the launcher
stack/	YOUR code (gitignored)
starter_kit/feb_driver/	the template you copy into stack/
tracks/	track folders
~/feb-sim/app/<os>/	the simulator app, downloaded by setup
Docker	runs the devkit container

The loop: the container mounts `stack/` and builds it; the bridge turns the simulator's stream into ROS 2 topics under `/autodrive/roboracer_1/`; your node reads the sensors and publishes two floats, `throttle_command` and `steering_command`, both in `[-1,1]`; the bridge sends them to the simulator; the car moves.

1 Install, once

1. **Docker.** `sudo apt install docker.io`. Then `sudo usermod -aG docker $USER` and log out and back in, so that `docker ps` works without `sudo`. Docker is a background service here; it starts at boot and nothing has to be opened. Check with `docker run -rm hello-world`.
2. **Python 3, git, curl, unzip.** `sudo apt install python3 git curl unzip`.
3. **GitHub CLI.** On Ubuntu 24.04, `sudo apt install gh`. On 22.04 the packaged version is too old: follow the three lines at https://github.com/cli/cli/blob/trunk/docs/install_linux.md. Then `gh auth login`.
4. **Foxglove.** Download the `.deb` from <https://foxglove.dev/download> and `sudo apt install ./foxglove-studio-*.deb`; open it once, sign in with a free account.

The simulator needs Vulkan. Intel and AMD GPUs work with the stock Mesa drivers; NVIDIA needs the proprietary driver (`sudo ubuntu-drivers install`). Run from a desktop session, not over plain SSH, so the app has a display.

2 Clone and set up

```
git clone https://github.com/Pranman1/feb-racing
cd feb-racing
./feb-sim setup --team "Your Team"
```

Pulls the devkit image and downloads the Linux build into `~/feb-sim/app/`. A few minutes. If `./feb-sim run` later fails with a Vulkan error, the GPU driver is the thing to fix; `vulkaninfo -summary` (package `vulkan-tools`) shows what is found.

3 Drive manually

```
./feb-sim run
```

Arrow keys drive. Menu top-left: Manual/Autonomous, Track, Cars, Look, camera. Close the window to stop. Nothing drives by itself without a stack.

4 Drive autonomously with the starter kit

```
cp -r starter_kit/feb_driver stack/
./feb-sim run --stack "ros2 launch feb_driver drive.launch.py"
```

The container builds the package, the simulator opens, the car drives, Foxglove opens on its topics.

What the starter driver is. A complete ROS 2 package. The driver is one commented file, `stack/feb_driver/feb_driver/reactive_driver.py`, a “follow the gap” controller: clean the lidar scan, blank a safety bubble around the nearest obstacle, aim at the widest gap, pick a speed from the room ahead, publish steering and throttle. Its numbers are in `stack/feb_driver/config/driver.yaml`.

Editing code. `stack/` is a plain folder on your machine; open it in any editor. With VS Code: code `stack/` from the repo folder, or File, Open Folder. Edit, save, run the same `./feb-sim run` command again; the container rebuilds your packages in seconds. No plugin or remote setup: the container reads the files straight from your disk.

5 Score and post

```
./feb-sim practice --stack "ros2 launch feb_driver drive.launch.py"
./feb-sim submit
```

`practice` runs one attempt the way a competition does: headless simulator, a warm-up lap plus ten timed laps, 10 s per collision, 300 s limit; it writes `runs/<id>/result.json` and a bag. `submit` opens a pull request with that file; a check validates and merges it, and your time appears on the site’s practice board as *unverified*. The organiser’s nightly rerun of your registered image on the same track turns it *verified*.

6 Looking inside: extra terminals and Foxglove

More terminals into the devkit

The container keeps running while the simulator is open. Open as many terminal windows as you like and, in each, from the repo folder:

```
./feb-sim shell
```

That drops you into a ROS 2 shell inside the container, with the environment already sourced. Useful commands there:

```
ros2 topic list                # everything on the graph
ros2 topic hz /autodrive/roboracer_1/lidar # is the sensor stream alive,
    at what rate
ros2 topic echo /autodrive/roboracer_1/throttle_command # what your node is sending
ros2 topic echo --once /autodrive/roboracer_1/imu
ros2 node list                 # your node should be here
ros2 node info /driver
ros2 param list /driver        # live parameters
ros2 bag record -a -o /home/autodrive_devkit/src/stack/mybag # lands in your stack/
    folder
```

`exit` leaves the shell; the container keeps running.

Devkit on its own. The launcher starts the devkit and the simulator together because that is what you want most of the time, but they are independent. The devkit alone, for building, replaying bags or running tests without a simulator:

```
docker run --rm -it -p 4567:4567 -v "$PWD/stack:/home/autodrive_devkit/src/stack" \
ghcr.io/pranman1/feb-devkit bash
```

Inside, `source install/setup.bash` after `colcon build`. The simulator alone is the app in `~/feb-sim/app/linux`, keyboard driving, no ROS. `docker exec -it feb-devkit bash` is the same thing without the launcher. Files you write under `/home/autodrive_devkit/src/stack` inside the container appear in `stack/` on your machine, and the other way round.

Foxglove

Installed in step 2 of the install list; sign in once. After that, `./feb-sim run` opens Foxglove connected to your car a few seconds after the simulator window. If it does not, or you want a second window: in Foxglove choose *Open connection, Foxglove WebSocket*, address `ws://localhost:8765`.

Panels worth adding (the `+` at the top right of a panel):

- **3D**: add the topic `/autodrive/roboracer_1/lidar`; set the display frame to the lidar's frame. The scan appears as points around the car.
- **Image**: `/autodrive/roboracer_1/front_camera`.
- **Plot**: `throttle_command.data` and `steering_command.data` against time; add `left_encoder.position` to watch the wheel.
- **Raw Messages**: any topic, to read exact values.
- **Topics** (left sidebar): every topic with its rate; a dash means nothing is publishing.

Save the layout (*Layout, Save*) and it comes back next time. Foxglove also opens the bags that practice records under `runs/<id>/`, and any bag you record yourself: *Open local file*.

7 When something goes wrong

```
docker logs feb-devkit      # build errors, your node's output
./feb-sim shell             # a ROS 2 terminal inside the container
./feb-sim stop              # kill everything and start over
```

Nothing drives: correct without `-stack`; with one, read `docker logs`. Port 4567 in use: `./feb-sim stop`. Docker pull denied, or “cannot connect to the Docker daemon”: the Docker service is stopped (`sudo systemctl start docker`) or you are not in the `docker` group yet.

8 Keeping up to date

```
./feb-sim update
```

One command pulls the repo (tracks, launcher, docs, new starter packages), pulls a newer devkit image if there is one, and replaces the simulator app when a new build is released. Your `stack/` folder is never touched. When a new starter package appears in `starter_kit/`, such as the cone-track driver, copy it the same way as the first one:

```
cp -r starter_kit/feb_cone_driver stack/
./feb-sim run --track loop_cones --stack "ros2 launch feb_cone_driver drive.launch.py"
```

9 What the car gives and takes

Topic (under /autodrive/roboracer_1/)	Meaning
lidar	LaserScan, 1080 beams over 270°, 0.06 to 10 m
imu	orientation, angular velocity, linear acceleration
left_encoder, right_encoder	wheel angle in rad; radius 0.059 m
front_camera	image
throttle_command, steering_command	you publish these: Float32 in $[-1, 1]$; throttle 0 is a hard brake; steering $\pm 1 = \pm 0.5236$ rad
ips, odom, lap and collision counters	ground truth: training only, barred at race time

Publish as many topics of your own as you like; only the two commands reach the car. Use message stamps or `use_sim_time`, never the wall clock, for speeds and integrals. Throttle about 0.1 is roughly 2 m/s.

A The starter package, file by file

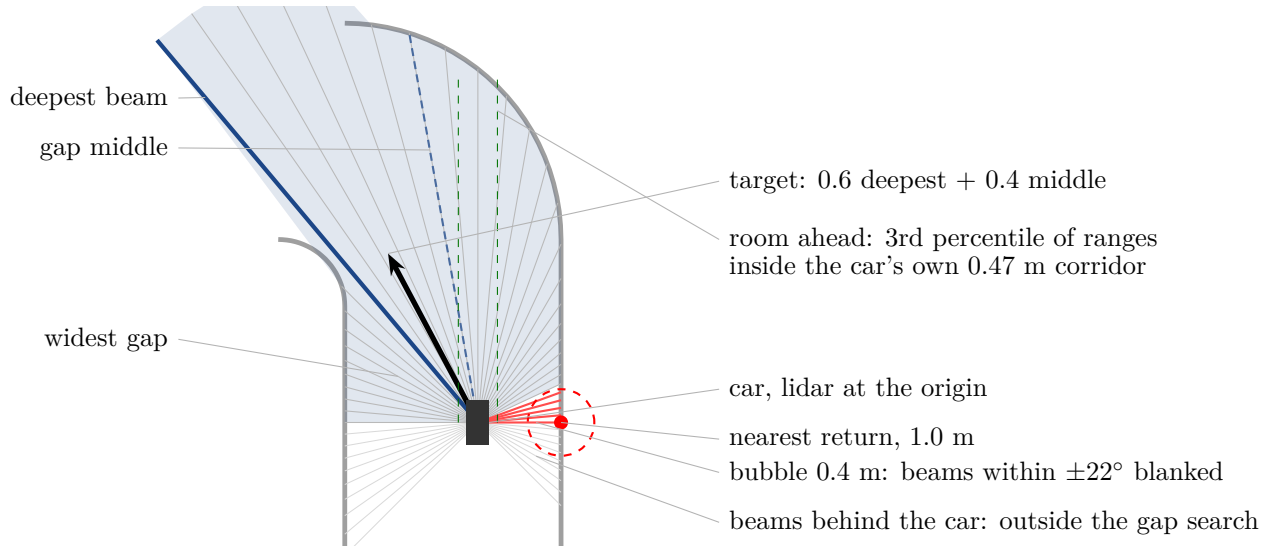
`starter_kit/feb_driver` is a normal ROS 2 Python package. After `cp -r starter_kit/feb_driver stack/` these are the files you own (the Python files live in the inner `feb_driver/` folder):

File	What it is
<code>package.xml</code>	The ROS 2 manifest: the package name <code>feb_driver</code> , its version and its dependencies (<code>rclpy</code> , <code>sensor_msgs</code> , <code>std_msgs</code>). <code>colcon</code> reads it to know the package exists. You only touch it to add a dependency.
<code>setup.py</code>	Python packaging. The line that matters is the entry point <code>reactive_driver = feb_driver.reactive_driver:main</code> : it is what makes <code>ros2 run feb_driver reactive_driver</code> exist. It also installs <code>launch/</code> and <code>config/</code> so launch files can find them. Add a line here for every new node.
<code>setup.cfg</code>	Two lines telling <code>colcon</code> where to put the installed scripts. Never edited.
<code>resource/feb_driver</code>	An empty marker file ROS 2 uses to index the package. Never edited.
<code>__init__.py</code>	Empty; makes the folder a Python module.
<code>reactive_driver.py</code>	The driver, about 170 lines, explained below.
<code>config/driver.yaml</code>	Every tuning number, one per line with a comment. The launch file hands it to the node, so you change behaviour here without touching code.
<code>launch/drive.launch.py</code>	Starts one node, <code>reactive_driver</code> , with the <code>yaml</code> and <code>use_sim_time</code> set, so the node's clock follows the simulator. This is what <code>-stack "ros2 launch feb_driver drive.launch.py"</code> runs.
<code>sysid_probe.py</code> , <code>launch/sysid.launch.py</code>	Stage 2. Drives with the same steering but a fixed throttle schedule (three steps, a coast, a chirp) while recording a bag, to measure how the car responds.
<code>tools/sysid_fit.py</code>	Fits that bag: steady-state gain (the <code>speed_per_throttle</code> number), idle brake, throttle lag and effective wheelbase. Run inside the container.

The node subscribes to three topics and publishes two, all under `/autodrive/roboracer_1/`: in, `lidar` (LaserScan, 1081 ranges over $\pm 135^\circ$) and `left_encoder`, `right_encoder` (JointState, wheel angle); out, `steering_command` and `throttle_command` (Float32). Steering is normalised, -1 to 1 meaning -30° to 30° . Throttle is 0 to 1 and **0 is a hard brake**, not coasting.

B How the driver drives, one scan at a time

Every lidar scan, about ten times a second, `on_scan` runs the steps below and publishes one steering and one throttle value. The figure shows one scan from above, in a corridor that bends left; the right wall is the nearest thing the lidar sees.



1. **Clean the scan.** Any beam that returned nothing (infinity) is set to the lidar's maximum range, and every range is clipped to the sensor's limits. This is the numpy array `ranges`; `angles` is the matching bearing of each beam.
2. **Find the nearest return and blank a bubble around it.** Only beams within $\pm \text{gap_fov}$ (1.6 rad, about $\pm 92^\circ$) count. The shortest of them is the nearest obstacle, the red dot. Every beam whose bearing is within $\arctan(\text{bubble}/r_{\min})$ of that beam is set to zero: at $r_{\min} = 1.0$ m and `bubble` = 0.4 m that is $\pm 22^\circ$, the red beams. A zeroed beam can never be chosen as a direction, so the car never aims through a gap that passes within 0.4 m of the closest thing it sees. That is the whole safety idea, and it is why the car brushes walls when `bubble` is too small and refuses corners when it is too big.
3. **Pick the widest gap.** The non-zero beams are split into runs of consecutive beams; the longest run is the gap (blue). Inside it, two candidates: the deepest beam (the longest range) and the middle beam of the run. The target bearing is a blend, $\text{deep_weight} \times \text{deepest} + (1 - \text{deep_weight}) \times \text{middle}$, 0.6/0.4 by default. Pure deepest cuts corners tight and hunts; pure middle is smooth but wide.
4. **Turn the bearing into steering.** The car is still turning at its current steering angle for the actuator delay (0.25 s) plus one scan period before the new command takes effect, so the heading change over that time is subtracted from the target first (delay compensation). Then `steer_gain` scales the bearing (0.7: a 20° target asks for 14° of steering), the result is clipped to the $\pm 30^\circ$ limit, low-passed with `steer_tau` and rate-limited to the actuator's 3.2 rad/s. Without the filtering, a target that jumps from one beam to the next would flick the wheel full lock every scan.
5. **Measure the room ahead.** Take the beams whose end point lies inside the car's own corridor (a strip 0.235 m either side of the centreline, the green dashes). The 3rd percentile of their forward distance is the clearance: not the minimum, which one noisy beam would spoil, but close to it. It is low-passed, except that a genuinely close obstacle (under `brake_margin` + 0.3 m) passes straight through so braking is immediate.

6. **Pick a speed.** Three ceilings: `max_speed` (2.5 m/s); the cornering speed $\sqrt{\text{lat_accel}/\kappa}$ from the slowly filtered steering, where $\kappa = \tan(\delta)/L$ is the path curvature; and the braking speed $\sqrt{2\text{decel_limit}(\text{clearance} - \text{brake_margin})}$, the fastest speed from which the car can still stop before the clearance. The goal is the smallest of the three, floored at `min_speed` (0.9 m/s) so the car never parks in a corner. The goal is low-passed and then slewed at `accel_limit` up and `decel_limit` down.
7. **Speed to throttle.** The wheel encoders give the measured speed (wheel angle change over the last 0.25 s times the wheel radius). Throttle is feed-forward plus trim: feed-forward is $v_{\text{target}}/\text{speed_per_throttle}$ (23 m/s per unit throttle, measured with the sysid tools, so 1 m/s needs about 0.043); trim is a small PI on the speed error, clipped to half of the feed-forward. Because of that clip the command can never reach zero while speed is wanted, and zero would be a hard brake. Finally the throttle is slew-limited to 0.8 per second. A textbook PI speed loop without these two guards surges and brakes in a 0.8 s cycle on this simulator; this one holds speed at 10 Hz.
8. **Publish.** Steering divided by 30° goes to `steering_command`, the throttle to `throttle_command`. On shutdown the node publishes 0,0 once so the car stops.

Follow mode (`follow_gap` > 0, used by the house racer as the opponent): if something sits within that distance in a $\pm 15^\circ$ cone straight ahead, it is treated as a car to sit behind, not a wall to steer around. Its beams are replaced by the corridor seen beside it, so the steering keeps the corridor's line, and the speed goal is capped to hold the distance.

Where to start tuning. `bubble` and `deep_weight` decide the line; `max_speed`, `lat_accel` and `brake_margin` decide how fast it dares. Change one number, run the same command, watch the lap time. The sysid tools are for when you want the throttle law to match the car rather than guess.